

High Level Design Document Template

A Web-Based Platform for Deploying AI Agents-Native Applications

Course: Workshop on Intelligent Systems and Autonomous Agents

Instructor: Dr. Gorge Kor

Submitted by:

- Shalev Yosef
- Nadav Simon
- Dmitry Todoseyev

Institution: The Academic College Of Tel Aviv Yaffo

Submission Date: 23/01/26

Why Base55 ?

The way consumers discover information and products is fundamentally changing. Traditional search engines are rapidly losing ground as users increasingly turn to conversational AI agents like Claude and ChatGPT. For businesses, this creates a new challenge: how do you visually showcase your products in a text-heavy AI environment? Project Base 55 is the bridge. Our platform allows anyone to simply input their existing API specifications, and we instantly generate a cutting-edge MCP (Model Context Protocol) server tailored to their business. Utilizing the new MCP and MCP UI standards, these automated servers render rich, interactive UI components directly within the AI host. We empower businesses to bypass complex integrations and effortlessly transform standard AI chats into dynamic, visual storefronts.

Little bit of terminology

REST API: The standard way software applications talk to each other over the internet. It allows a system to request data (like pulling a catalog of shoes) or trigger actions using standard web protocols.

API Spec (Specification): The blueprint or instruction manual for a REST API (like OpenAPI or Swagger). It tells developers—and our automated platform—exactly what endpoints exist, what data to send, and what format to expect in return.

LLM (Large Language Model): The foundational "brain" behind modern AI. It is a highly advanced model trained on massive amounts of data to understand, process, and generate human language.

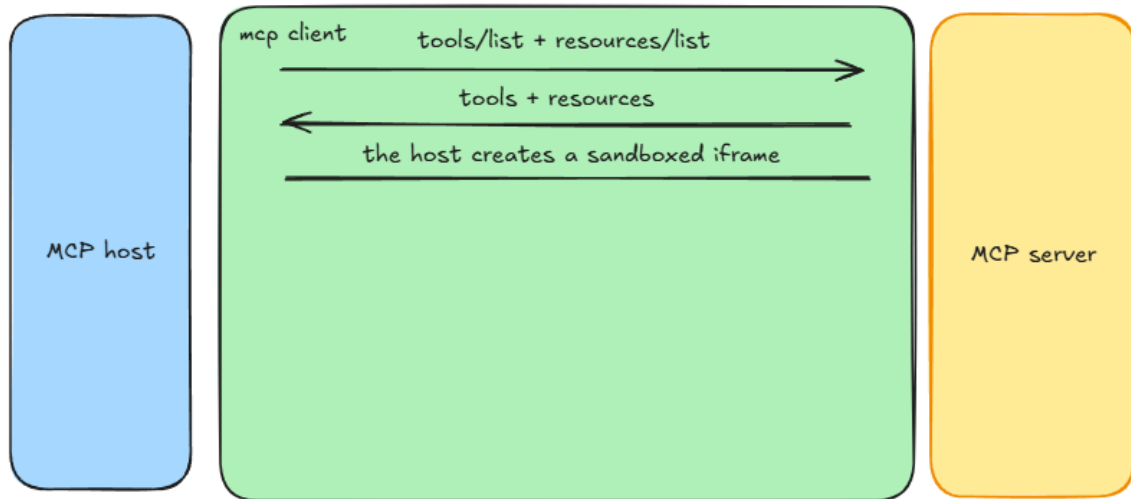
AI Agent: An autonomous AI system (powered by an LLM) that goes beyond just chatting. It can make decisions, use external tools, and take step-by-step actions to complete specific tasks for a user.

MCP Host: The application or interface where the user actually interacts with the AI agent (e.g., the Claude desktop app). It acts as the environment that can connect to outside tools to expand what the AI can do.

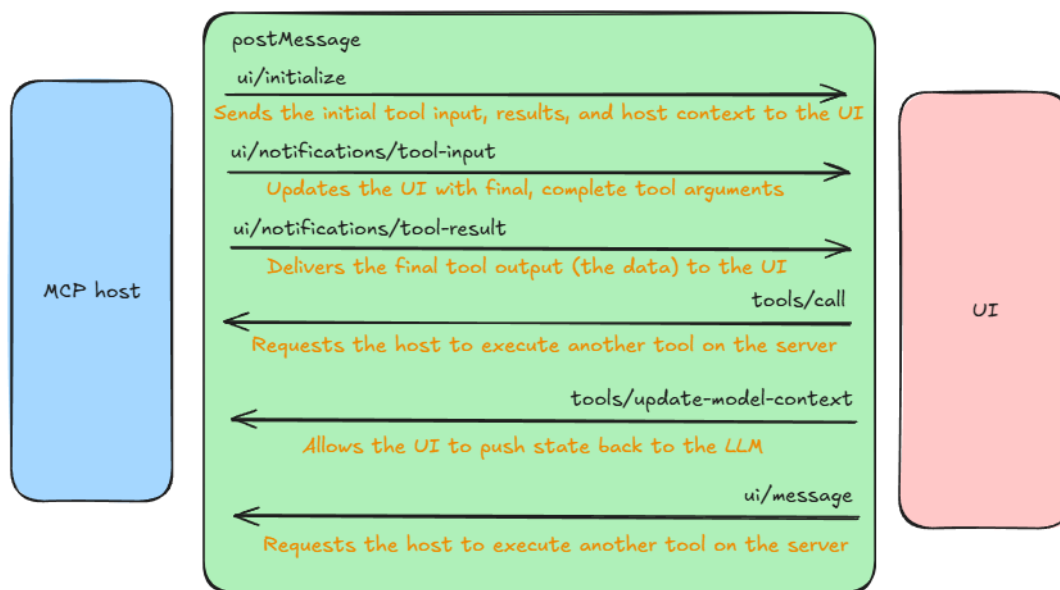
MCP Server (Model Context Protocol Server): A standardized connector that safely bridges the MCP Host with external data. In this project, it's the custom engine we automatically generate to translate the AI's requests into the customer's REST API calls.

MCP UI: A cutting-edge feature that allows the MCP Server to send back fully interactive, visual elements (like product carousels or buttons) instead of just plain text, rendering them directly inside the MCP Host's chat window.

MCP and MCP UI architecture

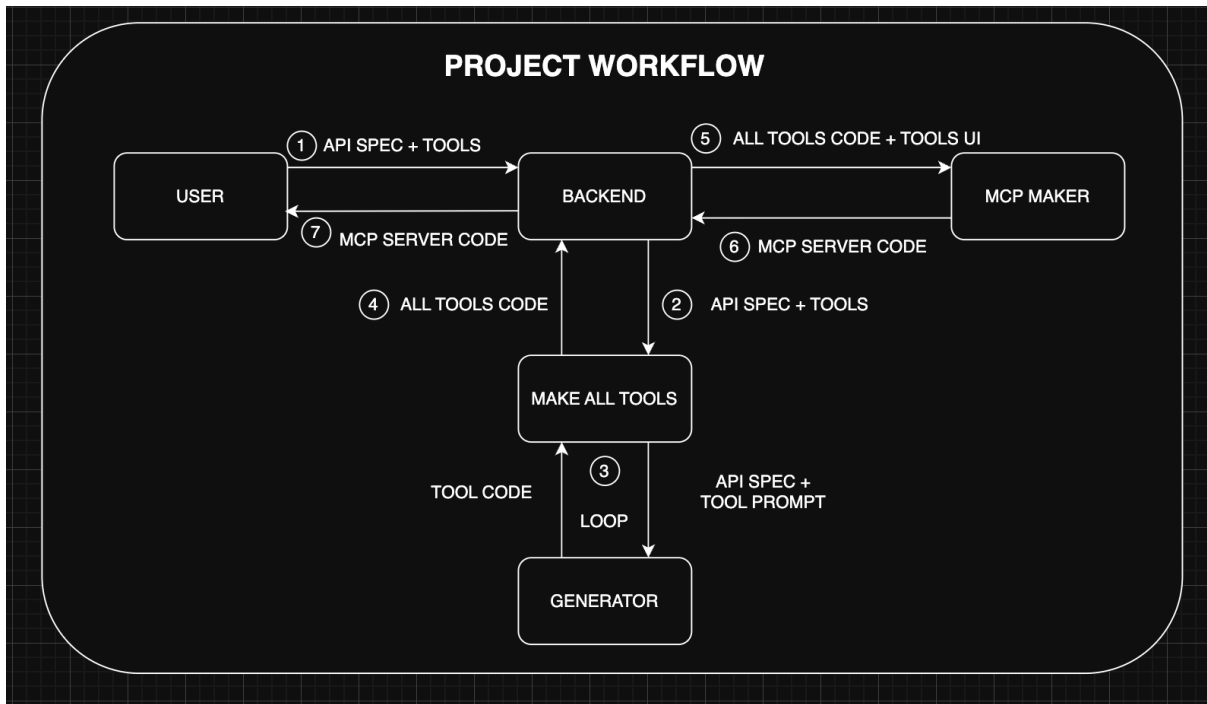


The diagram illustrates the initial connection and discovery sequence between an MCP host (which acts as the client) and an external MCP server. The process begins with the host sending a request to discover the server's capabilities, specifically asking for `tools/list + resources/list`. The MCP server responds by transmitting its available `tools + resources` back to the host. Following this underlying data exchange, the host creates a secure, isolated environment—a sandboxed iframe—which is used strictly to safely render the visual, interactive UI components provided by the server.



This communication model is "inverted" compared to traditional web development. The UI does not contact the company's server directly; instead, it sends a request to the host bridge, which then routes the request through the existing MCP connection to the server. This ensures that all actions are authenticated and governed by the host's policies.

The architecture relies on strict isolation and centralized control to safely display interactive elements. When an external service is connected, the system provides a secure, contained visual workspace. Rather than allowing this visual interface to communicate freely with outside networks as a standard web page typically would, every user interaction, data update, and button press is funneled entirely through the core AI application. By forcing all traffic through this central "middleman," the system guarantees that the primary application maintains complete oversight, verifying and controlling all data flow before any external actions are actually executed.



Step 1 (Input): The user provides their **API SPEC + TOOLS** to the system's **BACKEND**.

Step 2 (Routing): The **BACKEND** forwards the **API SPEC + TOOLS** to the **MAKE ALL TOOLS** module to begin processing.

Step 3 (The Generation Loop): The **MAKE ALL TOOLS** module communicates in a loop with the **GENERATOR**. It sends the **API SPEC + TOOL PROMPT** to the **GENERATOR**, which processes it and returns the individual **TOOL CODE**.

Step 4 (Aggregation): Once the loop is complete, the **MAKE ALL TOOLS** module sends the consolidated **ALL TOOLS CODE** back to the **BACKEND**.

Step 5 (Assembly): The **BACKEND** then sends the **ALL TOOLS CODE + TOOLS UI** to the **MCP MAKER**.

Steps 6 & 7 (Output): The **MCP MAKER** finalizes the build and sends the completed **MCP SERVER CODE** to the **BACKEND**, which ultimately delivers the final **MCP SERVER CODE** back to the user.

Components breakdown

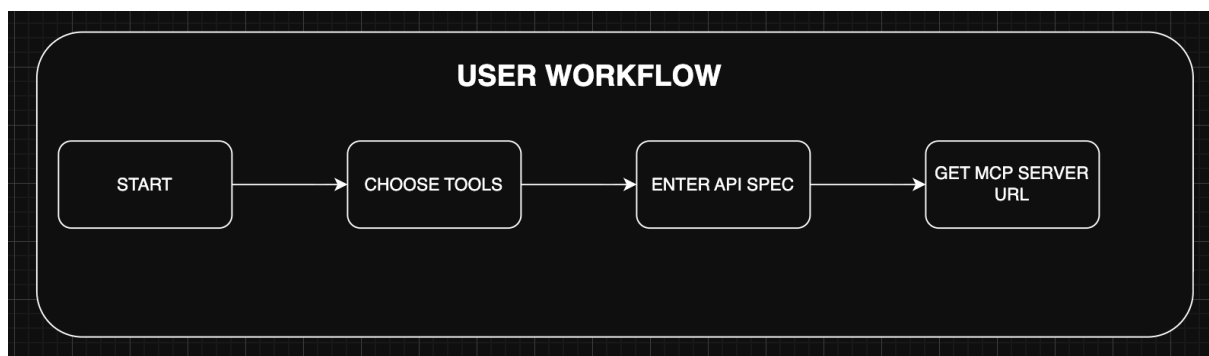
USER: The customer or developer interacting with the platform through a dedicated web interface. They upload their API specification and select their desired tools via the website, and ultimately receive their finished, ready-to-deploy MCP server code directly through the same web portal.

BACKEND: Our centralized server that orchestrates the entire pipeline. It receives the API spec and tool selections from the web frontend, routes the data into our generation engine, and later serves the finalized MCP server code back to the user's web session.

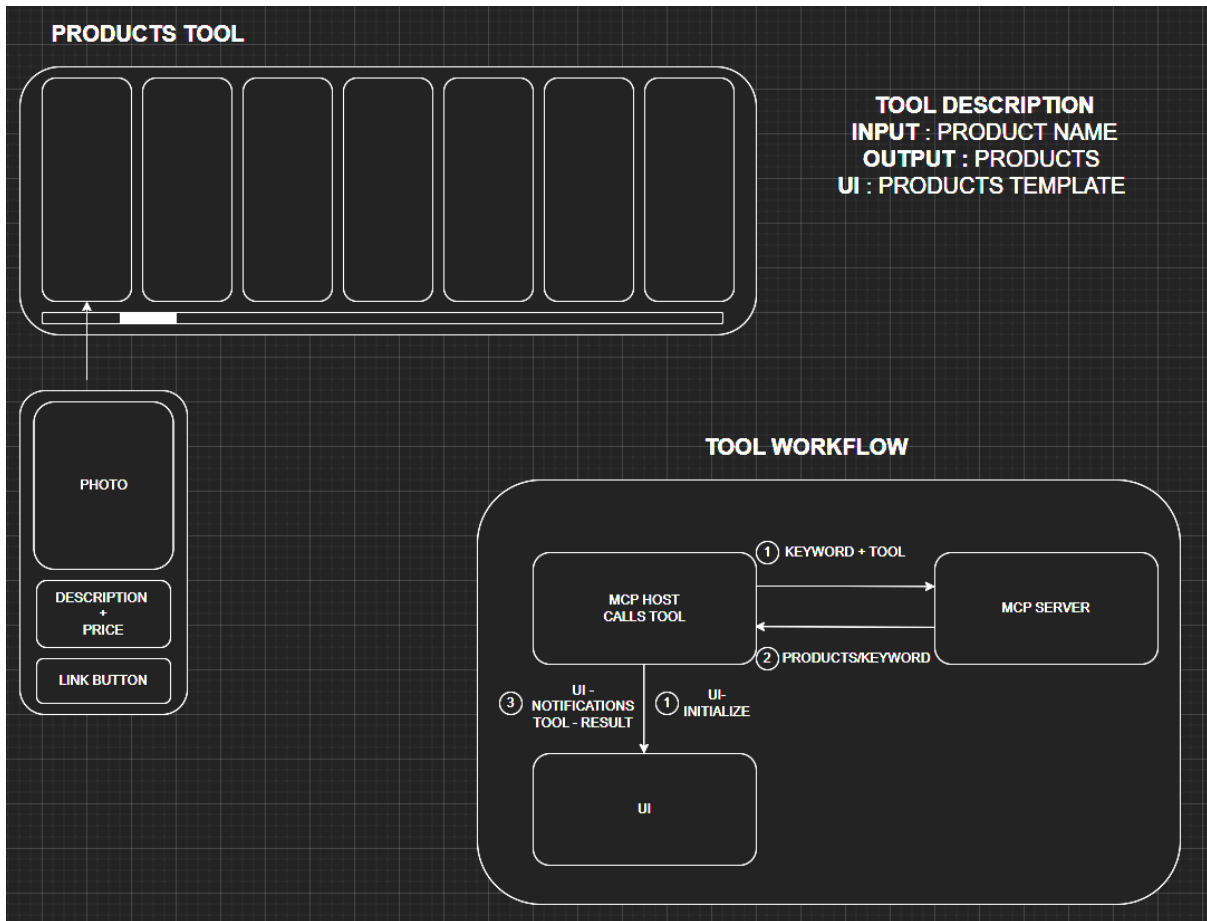
MAKE ALL TOOLS: The core orchestration function that drives the generation process. It contains an active loop that iterates through every specific tool the user chose to build. For each iteration, it bundles the API spec with a custom-engineered prompt—which includes a specialized template we created specifically for that tool—and sends it to the Generator. Once the loop finishes, it forwards the combined code for all tools to the MCP Maker.

GENERATOR (The LLM): Our AI-powered engine (a Large Language Model). Sitting inside the loop, it receives the API spec, the prompt, and our custom template, and intelligently writes the exact executable logic and code required for each individual tool.

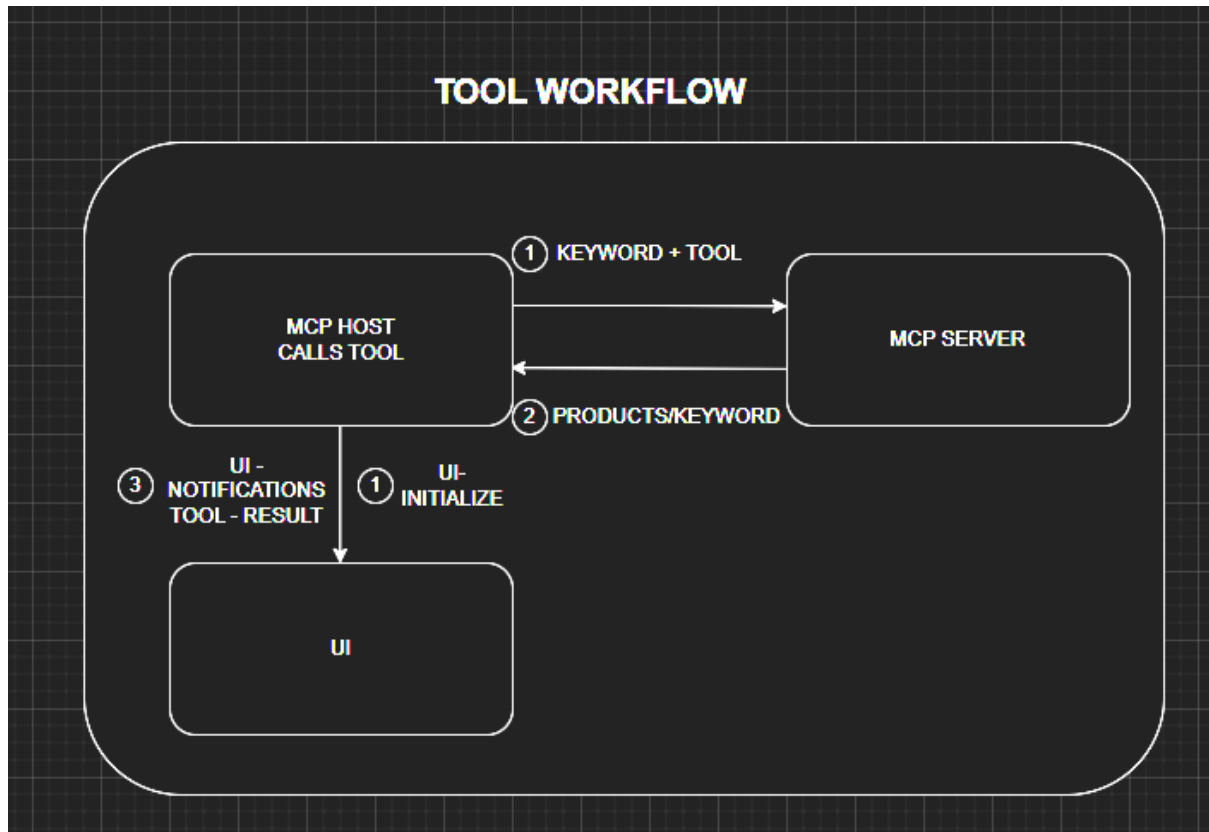
MCP MAKER: The final assembly function. Unlike the Generator, this is a deterministic, template-based function (not an LLM). It takes the collection of AI-generated tools and seamlessly renders them into a single, fully compliant MCP server package. It then returns this completed codebase to the Backend to be delivered to the user.



The process follows a straightforward, linear progression: it begins at the "Start" phase, after which the user selects their desired functionalities in the "Choose Tools" step. Following tool selection, the user inputs their existing application data structure during the "Enter API Spec" phase. The workflow seamlessly concludes when the system generates and provides a deployable link, represented by the final "Get MCP Server URL" step.



The UI Layout: A carousel-style template for displaying multiple items, where each individual product card is structured to contain a photo, a description with a price, and a link button. Each ui layout will have a template related to a specific tool. This ui layout is especially for presenting products.



Host to Server : The arrow labeled "(1) KEYWORD + TOOL" shows the MCP Host sending the initial execution command along with the user's specific search term (the keyword) to the external MCP Server.

Host to UI: This downward arrow demonstrates the host application signaling the user interface layer to set up and prepare the visual workspace or template.

Server to Host: The return arrow labeled "(2) PRODUCTS/KEYWORD" illustrates the MCP Server responding back to the host with the requested raw data, specifically the products that match the provided keyword.

Host to UI : This final downward arrow represents the host delivering the fetched product data directly into the user interface, allowing the visual template to be populated and rendered on the screen.

Technical Specification

1. Frontend Application (User Portal) The web interface where users upload API specifications, select tools, and retrieve their compiled MCP server URLs.

- **Framework:** Next.js (React)
- **Styling:** Tailwind CSS
- **API Parsing & Visualization:** `swagger-ui-react` (to render the uploaded OpenAPI/Swagger specifications visually for tool selection).

2. Backend Services (Orchestration & Routing) The central server responsible for managing the generation pipeline, receiving user inputs, and delivering the final artifact.

- **Runtime & Language:** Node.js with TypeScript (ensuring type safety across the orchestration loop and the generated code).
- **Web Framework:** Express.js
- **Spec Validation:** `@apidevtools/swagger-parser` (to programmatically extract endpoints, schemas, and parameters from the user's uploaded spec).

3. Generator (LLM Engine) The intelligence layer operating within the `Make All Tools` loop to dynamically write executable tool code based on custom prompts and API context.

- **LLM Integration:** OpenAI Node.js SDK (targeting GPT-4o for complex code generation).
- **Structured Output:** Zod (to enforce strict schema validation, ensuring the LLM outputs exactly the JSON or code structure required by the assembly loop).

4. MCP Maker (Assembly & Templating) The deterministic, non-LLM function responsible for combining the generated tool code and UI components into a complete, deployable server package.

- **Templating Engine:** Handlebars.js (used to inject the LLM-generated logic and raw UI HTML/CSS into a master MCP server blueprint)..

5. Output Artifact (The Generated MCP Server) The final standalone codebase delivered to the user, engineered to connect natively with AI hosts (e.g., Claude).

- **Core Protocol:** `@modelcontextprotocol/sdk` (the official Anthropic/MCP library to handle host-server handshakes, tool listing, and message routing)..
- **UI Components:** Vanilla HTML/CSS/JS (injected into the server output to be served to the host's sandboxed iframe for the MCP UI).

